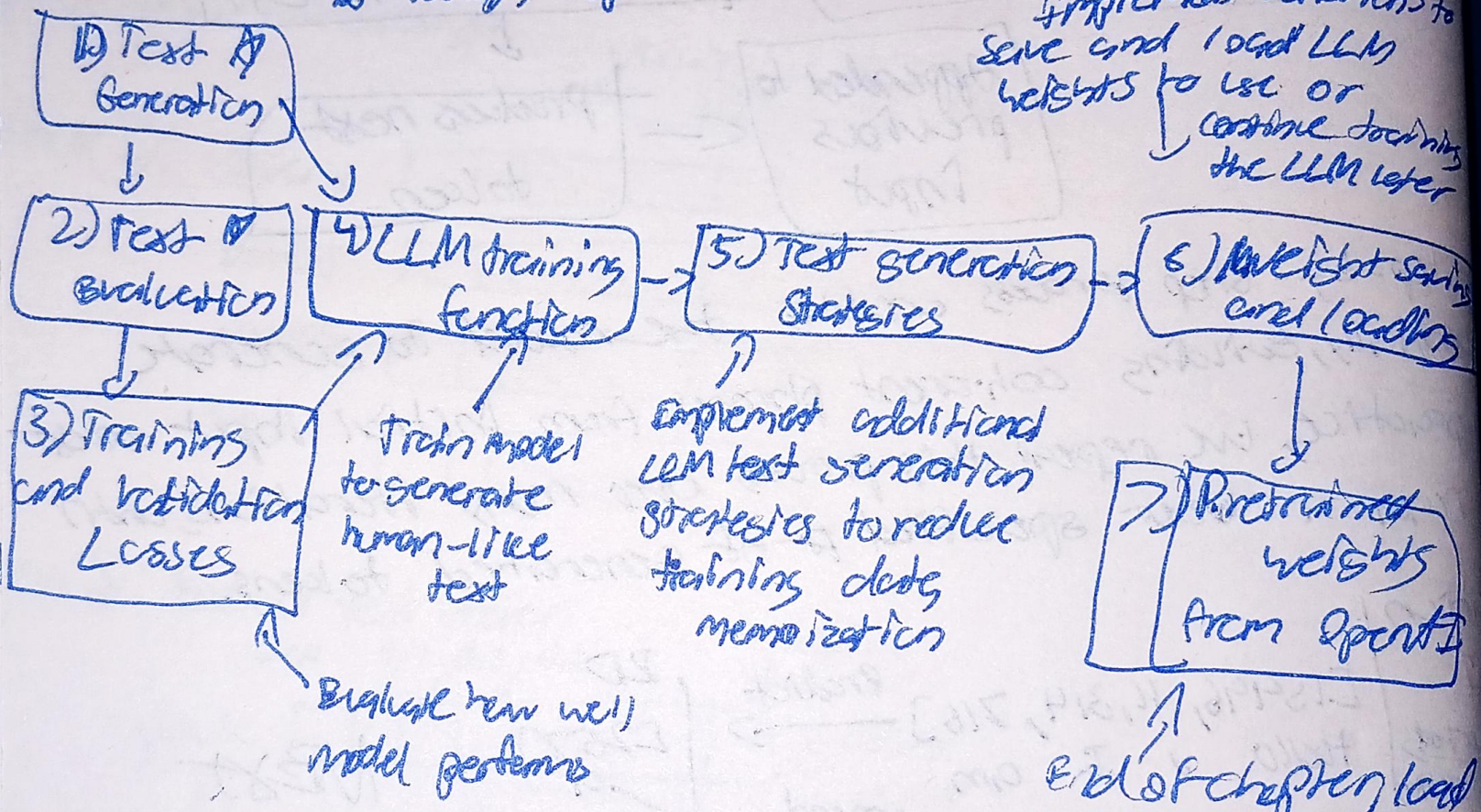
Pretraining LLMs: Loss Function

① How to measure LLM prediction loss?

= Today's topic



① Inputs and targets   Input → (Predicted ≈ Target) (current one target)

inputs = torch.tensor([[16833, 3626, 6100], # ['Every effort moves',
                       [40, 1107, 588]]) "I really like"]

Matching these inputs, the targets contain the token IDs we aim for the model to produce:

targets = torch.tensor([[3626, 6100, 345], # ['effort moves you',
                        [1107, 588, 11311]]) # 'really like chocolate']
↳ True prediction

② Outputs

Input → (GPT model) → Predicted

Step 1: Tokenizer converts tokens into token IDs
Step 2: GPT model converts token IDs into logits
Step 3: Logits are converted back into token IDs

Input token:

output Token IDs:

Token IDs:

"Every" tensor([[[16657],   → Batch 1
"Effort" [339],
"moves" [42826]],

"I" [[49906],
"really" [29669],   → Batch 2
"like" [41751]]])

Next PAGE →

③ Find loss between targets and output

Inputs

2) An untrained model produces random vectors for each token

Inputs		Vectors	Targets
every →	[2	[0.14, (0.14), 0.13, 0.17, 0.15, 0.13, 0.14]	effort → [1,
effort →	1,	[0.15, 0.13, 0.13, 0.16, (0.14), 0.15, 0.14]	moves → 4,
moves →	4]	[0.13, 0.14, 0.15, 0.16, 0.15, (0.13), 0.14]	you → 5]

index pos. 0 1 2 3 4 5 6
"a" "effort" "every" "forward" "moves" "you" "zero"

4) Goal is to maximize values that correspond to the index of token in target vector.

Target Indices:

targets = torch.tensor([[3626, 6100, 345], [1107, 588, 11311]])
i21 i22 i23

Batch 1: [i11, i12, i13]

Batch 2: [i21, i22, i23]

* Target Probabilities

i11 i12 i13

Batch 1: [p11, p12, p13]

i21 i22 i23

Batch 2: [p21, P22, P23]

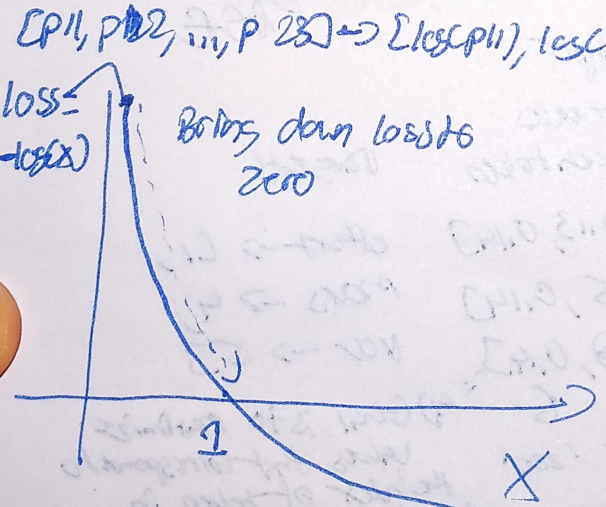
} Goal of training is to get these values as close to 1 as possible

↳ Merge them: [p11, p12, p13, p21, p22, p23]

1) Logits = [[[0.1113, -0.1057, -0.3666, ...,]]]
↓
2) Probabilities = [[[1.8849e-05, 1.5172e-05, 1.1687e-05, ...,]]]
↓
3) Target Probabilities = [[7.4541e-05, 1.5172e-05, 1.1687e-05, ...,]]]
↓
4) Log probabilities = [-9.5042, -10.3796, -11.3622, ...,]]]
↓
5) Average log probability = -10.7722
↓
6) Negative average log probability = 10.7722 ↙ Negative average log probability is the so-called loss we want to compute

↳ cross entropy loss (measures difference between 2 probability distributions)

[p11, p12, ..., p23] → [log(p11), log(p12) ... log(23)] ↗ mean

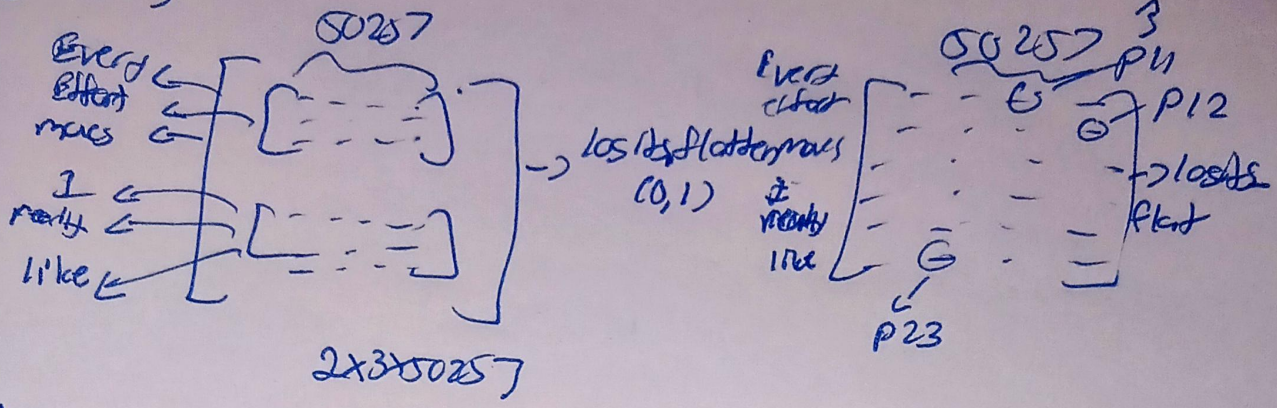


$= \frac{\sum \log(p11) + \dots \log(p23)}{6}$

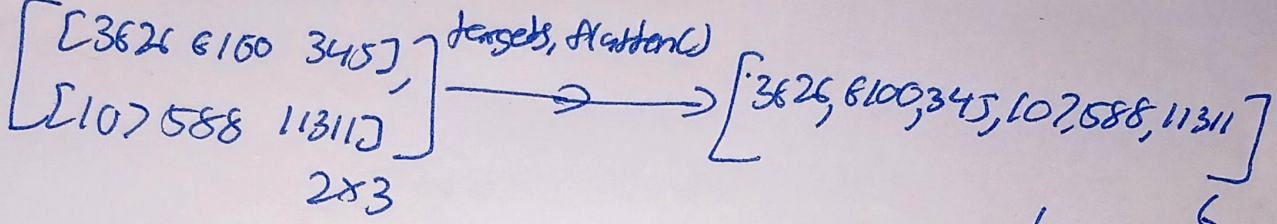
↳ $= -\frac{1}{6} \sum \log(p11) + \dots + \log(p23)$

↳ Negative log likelihood

Logits



Targets



torch.nn.functional.cross_entropy [logits_flat, targets_flat] Loss between adopt and target

① Softmax to logits

② Negative log likelihood

④ Perplexity: Another loss measure like cross entropy

↳ Measure how well the probability distribution predicted by the model matches the actual distribution of words in the dataset

↳ More interpretable way of understanding model uncertainty in predicting next token.

↳ Lower perplexity score = Better predictions

Perplexity = torch.exp(loss) e^loss 50257 = vocab_size

↳ Model is nearly as uncertain as if it had to choose next token randomly from about 46725 tokens in vocabulary

Much more interpretable